

Movie Watchlist Project Documentation

Field	Value
Student name	Bogdan Omelianenko
Student ID	2419079
Module	DSS2
Lecturer	Dimitar Kazakov
Project	Movie Watchlist

This document describes the Movie Watchlist full-stack application, including its purpose, technologies, data structure, authentication flow, CRUD logic, validation, testing, and setup instructions.

Project Overview

Movie Watchlist is a full-stack web application built around a private personal watchlist and a public community movie catalog. The application allows users to register, log in, manage their own protected watchlist items, and contribute movie records that can be viewed by the whole community.

The backend is an ASP.NET Core Web API running on port 4058. The frontend is a React and TypeScript application that consumes the backend API at <http://localhost:4058>. The system uses JWT authentication, DTOs, Entity Framework Core, SQLite, Cypress tests, and Docker support.

Technologies Used

Area	Technology	Purpose
Backend	ASP.NET Core Web API	Provides REST-style API endpoints for authentication, watchlist data, and community movies.
Backend	Entity Framework Core	Maps C# models to the SQLite database and handles queries.
Backend	SQLite	Stores users, private watchlist items, and public community movies in local development and Docker.
Backend	JWT Bearer Authentication	Protects private endpoints and identifies the current user.
Backend	Swagger / Swashbuckle	Provides API documentation and manual endpoint testing.
Frontend	React with TypeScript	Builds the user interface with typed components and routes.
Frontend	Vite	Runs the frontend development server and production build.
Frontend	Material UI	Provides the UI components used for forms, cards, buttons, alerts, and layout.
Testing	Cypress	Runs end-to-end tests against the real frontend and backend.
Deployment	Docker and Docker Compose	Runs backend and frontend containers with the backend exposed on port 4058.

Main Features

- User registration with email, display name, password, and confirm password.
- Login and logout functionality using JWT tokens.
- Current-user lookup through `/api/auth/me`.

- Protected personal watchlist page for authenticated users.
- Full create, read, update, and delete operations for the logged-in user's watchlist.
- Public community movie catalog that can be viewed without login.
- Authenticated users can create public community movie records.
- Only the creator of a community movie can edit or delete it.
- Backend DTOs and validation rules for all request and response bodies.
- Readable error messages displayed by the frontend.
- Cypress end-to-end test suite covering authentication, CRUD, validation, and authorization.
- Docker Compose setup with backend on `http://localhost:4058` and frontend on `http://localhost:5173`.

Project Structure

```
MovieWatchlist/  
  backend/  
    MovieWatchlist.Api/  
      Controllers/  
      Data/  
      Dtos/  
      Models/  
      Services/  
      Validation/  
      Program.cs  
      appsettings.json  
      Dockerfile  
  frontend/  
    src/  
      api/  
      components/  
      pages/  
      App.tsx  
      auth.tsx  
      types.ts  
    cypress/  
      package.json  
      cypress.config.ts  
      Dockerfile  
  docs/  
    development-log.md  
    demo-script.md  
  docker-compose.yml  
  README.md
```

Database Structure

The database is managed with Entity Framework Core and SQLite. The application contains three main entity models.

ApplicationUser

Field	Type	Description
Id	string	Unique user identifier generated as a GUID string.
Email	string	User email shown in safe user responses.
NormalizedEmail	string	Uppercase normalized email used for unique email lookup.
PasswordHash	string	PBKDF2 password hash. This field is never returned by the API.

DisplayName	string	Public display name used for user identity and community contributions.
CreatedAt	DateTime	UTC timestamp generated by the server.
WatchlistItems	collection	Private watchlist items owned by the user.
CommunityMovies	collection	Public community movie records created by the user.

WatchlistItem

Field	Type	Description
Id	int	Unique watchlist item identifier.
UserId	string	Owner user id. Used for private ownership checks.
Title	string	Required movie title.
Director	string?	Optional director.
ReleaseYear	int?	Optional release year between 1888 and current year + 2.
Genre	string?	Optional genre.
Status	string	Required status: PlanToWatch, Watching, Watched, or Dropped.
Rating	int?	Optional rating from 1 to 10.
Notes	string?	Optional private notes.
CreatedAt	DateTime	UTC creation timestamp.
UpdatedAt	DateTime	UTC update timestamp.

CommunityMovie

Field	Type	Description
Id	int	Unique community movie identifier.
CreatedByUserId	string	Id of the user who created the record.
CreatedByDisplayName	string	Display name saved for public attribution.
Title	string	Required public movie title.
Director	string?	Optional director.
ReleaseYear	int?	Optional release year between 1888 and current year + 2.
Genre	string?	Optional genre.
Description	string?	Optional public description.
AverageCommunityRating	decimal?	Optional extension field included in the model.
CreatedAt	DateTime	UTC creation timestamp.
UpdatedAt	DateTime	UTC update timestamp.

API Endpoints

Authentication Endpoints

Method	Endpoint	Auth Required	Purpose
POST	/api/auth/register	No	Creates a new user and returns a JWT plus safe user DTO.
POST	/api/auth/login	No	Authenticates a user and returns a JWT plus safe user DTO.
GET	/api/auth/me	Yes	Returns the current logged-in user based on the JWT token.

Protected Watchlist Endpoints

Method	Endpoint	Auth Required	Purpose
--------	----------	---------------	---------

GET	/api/watchlist	Yes	Returns the current user's private watchlist.
GET	/api/watchlist/{id}	Yes	Returns one watchlist item owned by the current user.
POST	/api/watchlist	Yes	Creates a new private watchlist item for the current user.
PUT	/api/watchlist/{id}	Yes	Updates one private watchlist item owned by the current user.
DELETE	/api/watchlist/{id}	Yes	Deletes one private watchlist item owned by the current user.

Public Community Movie Endpoints

Method	Endpoint	Auth Required	Purpose
GET	/api/community-movies	No	Lists all public community movie contributions.
GET	/api/community-movies/{id}	No	Returns details for one public community movie.
POST	/api/community-movies	Yes	Creates a new public movie contribution.
PUT	/api/community-movies/{id}	Yes	Updates a public movie only if the current user created it.
DELETE	/api/community-movies/{id}	Yes	Deletes a public movie only if the current user created it.

DTOs and Validation

The project uses DTOs for all request and response bodies. Controllers do not return EF Core database entities directly.

- RegisterRequestDto validates required email, email format, display name length, password length, and matching confirm password.
- LoginRequestDto validates required email and password fields.
- AuthResponseDto returns a JWT token and a safe UserResponseDto.
- CreateWatchlistItemDto and UpdateWatchlistItemDto validate title, director length, release year range, genre length, required status, rating range, and notes length.
- CreateCommunityMovieDto and UpdateCommunityMovieDto validate title, director length, release year range, genre length, and description length.
- Response DTOs return only safe fields that should be visible to the frontend.

Authentication Flow

1. A guest registers or logs in with email and password.
2. The backend normalizes the email and checks if the user exists.
3. During registration, the password is hashed with PBKDF2 using a random salt before it is stored.
4. During login, the submitted password is verified against the stored hash.
5. If authentication succeeds, the backend creates a JWT containing the user id, email, display name, subject, and token id claims.
6. The frontend saves the token in localStorage as movieWatchlistToken and saves the user DTO as movieWatchlistUser.
7. The API client automatically sends Authorization: Bearer <token> on authenticated requests.
8. Protected controllers read the current user id from the JWT claims to perform ownership checks.

- Logout removes the token and user data from localStorage.

Protected Watchlist CRUD Flow

- Create: an authenticated user submits the watchlist form. The backend creates a WatchlistItem with UserId taken from the JWT.
- Read: /api/watchlist filters records by the current user id, so users only see their own items.
- Update: the backend searches by both item id and current user id. If the record is not owned by the user, the API returns Not Found.
- Delete: the backend again checks both id and current user id before removing the item.
- The frontend allows users to add, edit, cancel editing, and delete items from the watchlist page.

Community Movie Contribution Flow

- Guests can view the community movie catalog and details without logging in.
- Authenticated users can create public community movie records.
- When a community movie is created, the backend saves CreatedByUserId and CreatedByDisplayName from the authenticated user.
- Update and delete operations check whether CreatedByUserId matches the current JWT user id.
- If a logged-in user tries to update or delete another user's community movie, the backend returns 403 Forbidden.
- The frontend shows edit and delete controls only for community movies created by the current user.

Frontend Pages

Page	Route	Description
Home Page	/	Introduces the private watchlist and public community movie sections.
Register Page	/register	Allows a new user to create an account.
Login Page	/login	Allows an existing user to authenticate. The demo account is pre-filled.
My Watchlist Page	/watchlist	Protected route for creating, editing, and deleting personal watchlist items.
Community Movies Page	/community	Public route for browsing community movies and authenticated contribution.

Frontend Implementation

- App.tsx defines the React Router routes and wraps the app with AuthProvider and Material UI ThemeProvider.
- auth.tsx stores and exposes the current user, JWT token, login, register, and logout functions.
- api/client.ts centralizes all API requests, base URL configuration, Authorization header handling, and error parsing.
- ProtectedRoute.tsx redirects guests away from protected pages.
- AppShell.tsx provides shared navigation and logout behavior.
- Form pages display loading states and backend validation errors through Material UI alerts.

Validation and Error Handling

- Required fields are enforced with DataAnnotations and custom validation.
- Email format is validated during registration.
- Password length must be at least 6 characters.
- Confirm password must match password.
- Display name must be 2 to 40 characters.
- Movie title must be 1 to 120 characters.
- Director may be up to 80 characters.
- Genre may be up to 40 characters.
- Watchlist notes may be up to 1000 characters.
- Community movie description may be up to 1500 characters.
- Release year must be between 1888 and current year + 2.
- Watchlist status must be one of PlanToWatch, Watching, Watched, or Dropped.
- Rating must be between 1 and 10 when provided.
- Invalid login returns 401 Unauthorized.
- Duplicate registration returns 409 Conflict.
- Unauthorized community movie modification returns 403 Forbidden.

Cypress Testing

The Cypress suite runs from the frontend project and tests the application as a real user would interact with it. It uses the real backend API at <http://localhost:4058>.

- Public community movie catalog can be viewed without login.
- Guest users cannot access the protected watchlist page.
- A user can register from the UI.
- A user can log in from the UI.
- An authenticated user can create, edit, and delete a watchlist item.
- An authenticated user can contribute a public community movie.
- A non-owner cannot edit or delete another user's community movie.
- Validation errors are displayed for invalid forms.

```
cd frontend
npm run test:e2e
```

```
# or interactive mode
npm run cypress:open
```

Dockerization

- The backend Dockerfile builds and publishes the ASP.NET Core API using .NET 8 images.
- The frontend Dockerfile builds the Vite application and serves the production build on port 5173.
- docker-compose.yml starts both services together.
- The backend is exposed on port 4058, as required.
- The frontend is exposed on port 5173.
- The SQLite database is stored in a Docker volume named backend-data.

```
docker compose up --build
```

Frontend: <http://localhost:5173>

Backend Swagger: <http://localhost:4058/swagger>

How to Run the Project Locally

The project can be opened in two separate VS Code windows: one for the backend and one for the frontend.

Backend

- Open the folder: MovieWatchlist/backend/MovieWatchlist.Api
- Make sure the .NET SDK is installed.
- Run the commands below.

```
dotnet restore
dotnet run --urls http://localhost:4058
```

The backend should be available at <http://localhost:4058> and Swagger should open at <http://localhost:4058/swagger>.

Frontend

- Open the folder: MovieWatchlist/frontend
- Make sure Node.js and npm are installed.
- Run the commands below while the backend is still running.

```
npm install
npm run dev
```

The frontend should be available at <http://localhost:5173>.

Demo Account

Email: alice@example.com
Password: Password123!

Development Log

- Analyzed the project requirements and separated the system into private watchlist data and public community movie data.
- Created the ASP.NET Core backend structure with Models, DTOs, Data, Services, Controllers, and Validation folders.
- Configured Entity Framework Core with SQLite and automatic database creation on startup.
- Implemented JWT registration, login, and current-user lookup.
- Implemented password hashing with PBKDF2 and random salts.
- Added protected CRUD endpoints for private watchlist items with current-user ownership filtering.
- Added public community movie endpoints with creator-only edit and delete logic.
- Integrated the React TypeScript frontend with the backend API URL <http://localhost:4058>.
- Added localStorage-based token handling and automatic Authorization headers.
- Created Cypress tests for authentication, CRUD, validation, and authorization behavior.
- Added Dockerfiles and docker-compose.yml for containerized execution.
- Prepared README, demo notes, and project documentation.

Problems Solved

- Implemented private ownership checks so users cannot access another user's watchlist.
- Implemented creator-only rules for community movie update and delete operations.
- Separated database entities from API responses using DTOs.
- Added readable backend validation errors that can be displayed by the frontend.
- Configured JWT token generation and validation with issuer, audience, signing key, and expiry.
- Configured CORS so the React frontend on port 5173 can call the backend on port 4058.
- Added seed data so the public catalog and demo login can be tested immediately.
- Configured Docker so the backend remains reachable at the required URL.

Conclusion

The Movie Watchlist project successfully implements a full-stack application that meets the main DSS2 resit requirements. It includes an ASP.NET Core Web API backend, Entity Framework Core with SQLite, JWT authentication, DTO-based validation, a React TypeScript frontend, protected personal watchlist CRUD, public community movie contribution, Cypress end-to-end tests, and Docker support with the backend running on port 4058.